ADITYA UNIVERSITY

SCHOOL OF COMPUTING
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Capstone project work entitled *"***University Academic Management Database Application***"* is submitted by

Y J P SARADHI	**(25B21CS086)**
G H D V S SURESH	**(25B11CS272)**
K BHUVANA HASINI	**(25B11CS401)**
D M S P SIDDHU SRI	**(25B11CS228)**

in partial fulfillment of the requirements for the award of the B.Tech. degree in Computer Science and Engineering for the academic year 2026-2027.

Project Guide
Mrs. K Vydehi, M.Tech.,(Ph.D),
Assistant Professor,
Department of CSE

Head of the Department
Dr. Phani Sridhar Addepalli, Ph.D,
Assistant Professor &HOD
Department of CSE

DECLARATION

We hereby declare that the Cornerstone project entitled "**University Academic Management Database Application**" is original and is submitted to **ADITYA UNIVERSITY**, Surampalem, in partial fulfillment of the requirements for the award of the **B.Tech. Degree** in **Computer Science and Engineering**. We further declare that this project work has not been submitted, either in full or in part, for the award of any degree in this or any other institution. We also confirm that this work complies with the Institutional Academic Integrity and AI Usage Policy

Place: Surampalem
Date:

By

Y J P SARADHI	(25B21CS086)
G H D V S SURESH	(25B11CS272)
K BHUVANA HASINI	(25B11CS402)
D M S P SIDDHU SRI	(25B11CS228)

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, **Mrs. K Vydehi, Assistant Professor,** who has guided us throughout the project. Her valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our Deputy Heads of the Department, **Dr. Chandra Sekhar Kolli, Associate Professor , Dr Hari Jyothula , Associate Professor and Dr Pennada Siva Satya Prasad, Assistant Profeesor,** Department of Computer Science and Engineering, for their support and valuable suggestions.

Our deepest thanks to our Head of the Department **Dr. Phani Sridhar Addepalli, Assistant Professor,** for his inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **Dr. M.V. Rajesh, Associate Dean, School of Computing** for their encouragement and support during the course of our project.

We would like to express our sincere gratitude to **Dr. G. Suresh, Registrar, Dr. A Ramesh , Pro Vice - Chancellor (Engineering & Sciences), Dr. S. Rama Sree , Pro Vice-Chancellor (Academics), Dr. M.B. Srinivas , Vice - Chancellor, Dr. M. Sreenivasa Reddy, Deputy Pro Chancellor and Management, Aditya University** for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the **faculty members, lab technicians, non-teaching staff, and our friends** who have directly or indirectly supported us in completing this Cornerstone project successfully.

TABLE OF CONTENTS

S.NO	TOPIC	PAGE NO.
1	Cover page	I
2	Certificate	II
3	Acknowledgement	III
4	Table of Contents	1
5	Abstract	2
6	List of figures	3
7	List of tables	4
8	Acronyms	5
9	Introduction	6
10	System Analysis	7
11	Database Design	9
12	ER Diagram	10
13	Relational Schema	11
14	Table Design	12
15	Normalization	14
16	SQL Implementation	17
17	SQL Queries & Screenshots	21
18	Conclusion and Future Enhancements	33
19	References	34

ABSTRACT

TITLE: COURIER MANAGEMENT SYSTEM

The Courier Management System is a database management project developed to efficiently manage and organize courier-related information. It maintains details of customers, couriers, delivery agents, payments, feedback, and tracking through a structured relational database. The system records courier bookings, including source, destination, receiver details, parcel weight, and booking date, while allowing delivery agents to be assigned and shipment statuses to be monitored. It also manages payment information and customer feedback for effective record management. The project implements the ER model using Oracle SQL, creates the required relational tables, and inserts sample records for testing. Various basic and advanced SQL queries involving selection, filtering, joins, aggregation, grouping, ordering, and subqueries are used to retrieve meaningful information. The database is normalized up to Third Normal Form (3NF) to reduce redundancy, prevent data anomalies, and improve data consistency and integrity.

LIST OF FIGURES

S.No.	Figure Name	Page No.
1	ER Diagram of Courier Management System	10
2	SQL Queries and Screenshots	21-32

LIST OF TABLES

S.No.	Table Name	Page No.
1	Major Database Entities	9
2	Customer Table Structure	12
3	Courier Table Structure	12
4	Payment Table Structure	12
5	Tracking Table Structure	12
6	Delivery Agent Table Structure	13
7	Keys and Constraints	13
8	Sample Data Records	14

ACRONYMS

- DBMS – Database Management System
- ER – Entity Relationship
- SQL – Structured Query Language
- DDL – Data Definition Language
- DML – Data Manipulation Language
- DQL – Data Query Language
- DCL – Data Control Language
- TCL – Transaction Control Language
- PK – Primary Key
- FK – Foreign Key
- 1NF – First Normal Form
- 2NF – Second Normal Form
- 3NF – Third Normal Form

1. INTRODUCTION

1.1 Brief Information about the Project

The Courier Management System is a DBMS-based application for organizing information handled by a courier service. It maintains customer details, courier bookings, receiver information, source and destination addresses, shipment status, delivery agents, payments, and tracking updates in related database tables.

1.2 Motivation and Contribution of Project

Courier operations involve many records and frequent updates. Manual registers or disconnected files can make searching, updating, and maintaining consistent records difficult. A centralized relational database provides a systematic way to store related information and connect records through primary and foreign keys.

- Centralizing academic records in one relational database.
- Reducing duplicate and inconsistent information.
- Maintaining referential integrity using primary and foreign keys.
- Supporting faster courier searches and tracking.
- Providing a foundation for booking, delivery, payment, and reporting.

1.3 Objectives of the Project

- Develop a centralized courier database.
- Manage customer records.
- Maintain payment information.
- Store courier and parcel details.
- Track shipment status and location.
- Assign delivery agents.
- Reduce duplicate data and update problems.
- Support efficient SQL queries and reports.

1.4 Scope of the Project

The scope covers customer, courier, payment, tracking, and delivery-agent information. The database can later be extended with notifications, route optimization, online payment integration, mobile interfaces, and advanced reports.

2. SYSTEM ANALYSIS

2.1 Existing System

- Manual record maintenance.
- Multiple files or spreadsheets.
- Duplicate records and inconsistent information.
- Time-consuming searches.
- Difficulty in generating consolidated reports.
- Higher possibility of data-entry errors.

2.2 Proposed System

The proposed system uses a centralized relational database. It organizes courier information into related tables and uses primary keys, foreign keys, uniqueness rules, NOT NULL conditions, and CHECK constraints where appropriate.

- Centralized database
- Customer management
- Courier booking management
- Payment management
- Shipment tracking
- Delivery-agent assignment
- SQL-based reporting
- Referential integrity

2.3 Functional Requirements

Module	Functional Requirement
Customer Management	Add, update, search, and display customer details such as Customer ID, name, phone, email, and address.
Courier Management	Add and maintain courier details such as Courier ID, Customer ID, receiver name, receiver phone, source, destination, weight, and booking date.
Payment Management	Store and maintain payment details including Payment ID, Courier ID, amount, payment mode, and payment status.
Tracking Management	Track courier details using Tracking ID, Courier ID, location, status, and updated date.
Delivery Agent Management	Add, update, search, and display delivery agent details such as Agent ID, name, phone, and area.
Feedback Management	Store and maintain customer feedback including Feedback ID, Customer ID, Courier ID, rating, and comments.

2.4 Non-Functional Requirements

- Performance – Queries should return customer, courier, payment, tracking, and delivery information efficiently for normal workloads.
- Integrity – Invalid references and duplicate key values should be prevented, and relationships between customers, couriers, payments, tracking, and delivery agents should remain consistent.
- Security – Database access and privileges should be controlled so that only authorized administrative users can modify important records.
- Usability – Forms, tracking information, and reports should be simple, clear, and understandable to customers and staff.
- Reliability – Transactions should preserve a consistent database state and ensure that courier, payment, and tracking records are stored correctly.
- Scalability – The database design should support growth in customers, courier bookings, payments, tracking records, and delivery agents without affecting normal operations.

2.5 Requirements Specification

Category	Suggested Requirement
Hardware	Intel i3 or above; 4 GB RAM minimum; adequate storage; standard display and keyboard/mouse.
Operating System	Windows 10/11 or an equivalent supported platform.
Database	Oracle Database or another relational DBMS used by the implementation.

3. DATABASE DESIGN

3.1 Major Entities

Entity	Description
Customer	Stores customer information.
Courier	Stores booking, parcel, sender/receiver, and destination information.
Payment	Stores payment information related to a courier.
Tracking	Stores shipment location and status updates.
Delivery Agent	Stores delivery personnel details and assigned delivery work.
Feedback	Stores customer feedback, ratings, and comments related to a courier.

3.2 ER Model

The ER model represents the six major entities and their relationships. The main relationships are Customer–Courier, Courier–Payment, Courier–Tracking, Delivery Agent–Courier, and Customer–Feedback/Courier–Feedback.

3.3 Relationships

Relationship	Description
Customer → Courier	One customer can book many courier shipments.
Courier → Payment	One courier booking has one payment record in the project design.
Courier → Tracking	One courier can have multiple tracking updates.
Delivery Agent → Courier	One delivery agent can handle multiple courier deliveries.
Customer → Feedback	One customer can provide multiple feedback records.
Courier → Feedback	One courier can have one feedback record in the project design.

4. ER DIAGRAM

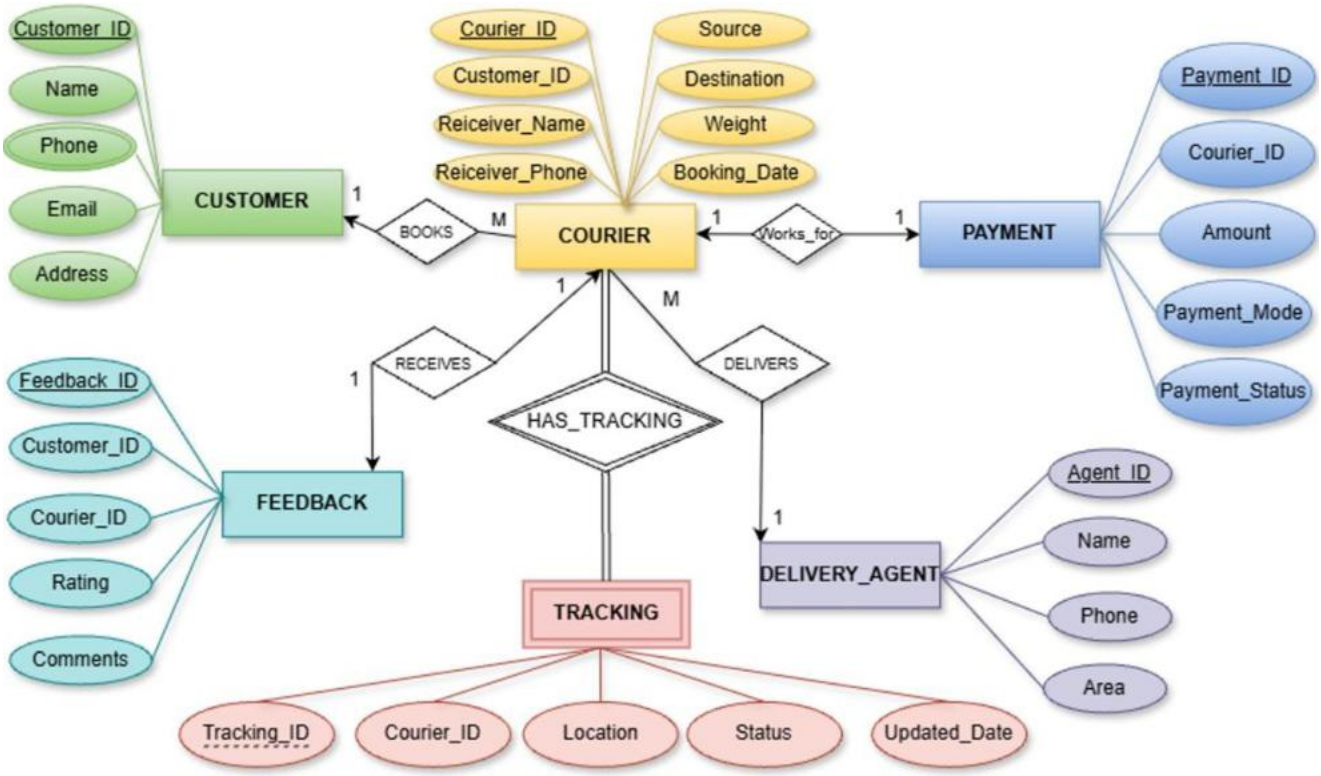


Figure 1: ER Diagram of Courier Management System

The ER diagram summarizes the entity structure and cardinalities described in the supplied DBMS project material. It is a documentation-level representation; the final project team should align attribute names and cardinalities with the actual implemented database.

5. RELATIONAL SCHEMA

Relation	Relational Schema
CUSTOMER	Customer_ID (PK), Name, Phone, Email, Address
COURIER	Courier_ID (PK), Tracking_ID, Customer_ID (FK), Receiver_Name, Receiver_Phone, Source, Destination, Weight, Booking_Date, Status
PAYMENT	Payment_ID (PK), Courier_ID (FK), Amount, Payment_Mode, Payment_Status
TRACKING	Tracking_ID (PK), Courier_ID (FK), Location, Status, Updated_Date
DELIVERY_AGENT	Agent_ID (PK), Name, Phone, Area
FEEDBACK	Feedback_ID (PK), Customer_ID (FK), Courier_ID (FK), Rating, Comments

The Customer relation stores customer details. Courier references Customer through Customer_ID. Payment and Tracking reference Courier through Courier_ID. Delivery Agent stores delivery-person information. The relations convert the ER design into database tables.

6. TABLE DESIGN

6.1 Customer Table

Attribute	Data Type	Constraint	Description
Customer_ID	INT	PK, NOT NULL	Unique customer identifier
Name	VARCHAR	NOT NULL	Customer name
Phone	VARCHAR	–	Contact number
Email	VARCHAR	UNIQUE	Customer email
Address	VARCHAR	–	Customer address

6.2 Courier Table

Attribute	Data Type	Constraint	Description
Courier_ID	INT	PK	Unique courier identifier
Tracking_ID	VARCHAR	UNIQUE	Shipment tracking identifier
Customer_ID	INT	FK	References Customer
Receiver_Name	VARCHAR	NOT NULL	Receiver name
Receiver_Phone	VARCHAR	—	Receiver contact
Source	VARCHAR	—	Pickup location
Destination	VARCHAR	–	Delivery location
Weight	DECIMAL	CHECK	Parcel weight
Booking_Date	DATE	—	Booking date
Status	VARCHAR	—	Current shipment status

6.3 Payment Table

Attribute	Data Type	Constraint	Description
Payment_ID	INT	PK	Unique payment identifier
Courier_ID	INT	FK	References Courier
Amount	DECIMAL	CHECK	Payment amount
Payment_Mode	VARCHAR	—	Payment mode
Payment_Status	VARCHAR	–	Payment status

6.4 Tracking Table

Attribute	Data Type	Constraint	Description
Tracking_ID	VARCHAR	PK	Unique tracking identifier
Courier_ID	INT	FK	References Courier
Location	VARCHAR	–	Tracking location
Status	VARCHAR	–	Shipment status
Updated_Date	DATE	—	Tracking update date

6.5 Delivery Agent Table

Attribute	Data Type	Constraint	Description
Agent_ID	INT	PK	Unique agent identifier
Name	VARCHAR	NOT NULL	Agent name
Phone	VARCHAR	—	Agent contact
Area	VARCHAR	—	Assigned area

6.6 Keys and Constraints

Constraint Type	Applied To / Purpose
Primary Key	Customer_ID, Courier_ID, Payment_ID, Tracking_ID, Agent_ID
Foreign Key	Courier.Customer_ID; Payment.Courier_ID; Tracking.Courier_ID
NOT NULL	Required names and identifiers
UNIQUE	Tracking ID and customer email where applicable
CHECK	Weight and payment amount can be restricted to valid values

7. NORMALIZATION

Normalization organizes data into related relations to reduce redundancy and update anomalies. The Courier Management System is normalized from the Unnormalized Form (UNF) up to the Third Normal Form (3NF). The process separates customer, courier, delivery agent, payment, feedback, and tracking information into related relations.

7.1 Unnormalized Form (UNF)

A single courier record may contain Customer, Courier, Delivery Agent, Payment, Feedback, and Tracking information together. If the same customer books multiple couriers, customer details and related information may be repeated. Similarly, delivery-agent and payment information can be repeated across courier records, creating redundancy.

7.2 First Normal Form (1NF)

In 1NF, repeating groups are removed and each attribute contains a single atomic value. Each customer, courier, delivery agent, payment, feedback, and tracking value is stored separately rather than as repeating groups within one record. However, customer and delivery-agent details may still be repeated across multiple courier records.

COURIER_ID	CUSTOMER_ID	CUSTOMER_NAME	AGENT_ID	AGENT_NAME	PAYMENT_ID	AMOUNT	TRACKING_ID	LOCATION	STATUS
1001	1	RAVI	101	RAM	P501	450	T001	DELHI	DELIVERED
1002	2	PRIYA	102	SRINU	P502	300	T002	HYDERABAD	IN TRANSIT
1003	3	ARUN	103	MAHI	P503	600	T003	CHENNAI	DELIVERED

PRIMARY KEY: COURIER_ID

7.3 Second Normal Form (2NF)

In 2NF , partial dependencies on a composite key are removed. Courier-specific attributes depend on Courier_ID, customer-specific attributes depend on Customer_ID, and delivery-agent-specific attributes depend on Agent_ID. Payment, feedback, and tracking information are separated into their respective relations. This decomposition reduces repeated information and improves data consistency.

PAYMENT

PAYMENT_ID	COURIER_ID	AMOUNT	PAYMENT_MODE	PAYMENT_STATUS
P501	1001	450	UPI	PAID
P502	1002	300	CASH	PENDING
P503	1003	600	CARD	PAID

PRIMARY KEY: PAYMENT ID FOREIGN KEY: COURIER ID

DELIVERY_AGENT

AGENT_ID	NAME	PHONE	AREA
101	RAM	9479201746	HYDERABAD
102	SRINU	9830847203	VIJAYAWADA
103	MAHI	9082746164	WARRANGAL

PRIMARY KEY: AGENT_ID

7.4 Third Normal Form (3NF)

In 3NF, transitive dependencies are removed. Customer details are stored in CUSTOMER rather than being repeated in COURIER . Delivery - agent details are stored in DELIVERY_AGENT , while COURIER stores the corresponding Agent_ID reference. Payment, feedback, and tracking details are stored in their respective relations. The resulting structure contains the six related relations and satisfies the 3NF objective.

CUSTOMER

CUSTOMER_ID	NAME	PHONE	EMAIL	ADDRESS
1	RAVI	9876543210	ravi@gmail.com	HYDERABAD
2	PRIYA	9876543211	priya@gmail.com	VIJAYAWADA
3	ARUN	9876543212	arun@gmail.com	WARANGAL

Relation	Reason for Separation
CUSTOMER	Stores customer-specific details such as name, phone, email, and address once.
DELIVERY_AGENT	Stores delivery-agent-specific details such as name, phone, and area once.
COURIER	Stores courier-specific details and references the customer and delivery agent.
PAYMENT	Stores payment information associated with a courier.
FEEDBACK	Stores customer feedback, rating, and comments associated with a courier.

8. SQL IMPLEMENTATION

8.1 DDL – Data Definition Language

The following DDL is a documentation-ready implementation aligned with the supplied relational schema. Data types may be adjusted to the specific DBMS used by the student team.

```
CREATE TABLE Customer (
Customer_ID INT PRIMARY KEY,
Name VARCHAR(100) NOT NULL,
Phone VARCHAR(15),
Email VARCHAR(100) UNIQUE,
Address VARCHAR(200)
);

CREATE TABLE Courier (
Courier_ID INT PRIMARY KEY,
Tracking_ID VARCHAR(30) UNIQUE,
Customer_ID INT,
Receiver_Name VARCHAR(100) NOT NULL,
Receiver_Phone VARCHAR(15),
Source VARCHAR(100),
Destination VARCHAR(100),
Weight DECIMAL(8,2),
Booking_Date DATE,
Status VARCHAR(30),
FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
);

CREATE TABLE Payment (
Payment_ID INT PRIMARY KEY,
Courier_ID INT,
Amount DECIMAL(10,2),
Payment_Mode VARCHAR(30),
Payment_Status VARCHAR(30),
FOREIGN KEY (Courier_ID) REFERENCES Courier(Courier_ID)
);
```

8.2 DML – Data Manipulation Language

-- Insert Customer

```
INSERT INTO Customer (Customer_ID, Name, Phone, Email, Address)
VALUES (101, 'Rahul Kumar', '9876543210', 'rahul@example.com', 'Hyderabad');
```

-- Insert Courier

```
INSERT INTO Courier (Courier_ID, Tracking_ID, Customer_ID, Receiver_Name, Receiver_Phone,
 Source, Destination, Weight, Booking_Date, Status)
VALUES (301, 'TRK1001', 101, 'Priya Sharma', '9876501234',  'Hyderabad', 'Vijayawada',
2.5, '2026-09-28', 'Booked');
```

-- Insert Payment

```
INSERT INTO Payment  (Payment_ID, Courier_ID, Amount, Payment_Mode, Payment_Status)
VALUES (401, 301, 250.00, 'UPI', 'Paid');
```

-- Insert Tracking

```
INSERT INTO Tracking  (Tracking_ID, Courier_ID, Location, Status, Updated_Date)
VALUES  ('TRK1001', 301, 'Hyderabad', 'In Transit', '2026-09-28');
```

-- Insert Delivery Agent

```
INSERT INTO Delivery_Agent (Agent_ID, Name, Phone, Area)
VALUES (501, 'Arun Kumar', '9876512345', 'Vijayawada');
```

-- Insert Feedback

```
INSERT INTO Feedback (Feedback_ID, Customer_ID, Courier_ID, Rating, Comments)
VALUES (601, 101, 301, 5, 'Good and fast delivery');
```

8.3 DQL – Data Query Language

1. Display all customers

```
SELECT *FROM Customer;
```

2. Display all courier details

```
SELECT *
FROM Courier;
```

3. Display courier details of a particular customer

```
SELECT Courier_ID, Tracking_ID, Receiver_Name, Destination, Status
FROM Courier
WHERE Customer_ID = 101;
```

4. Display couriers ordered by destination

```
SELECT Courier_ID, Tracking_ID, Source, Destination, Weight
FROM Courier
ORDER BY Destination;
```

5. Display customer and courier details

```
SELECT c.Name, co.Tracking_ID, co.Receiver_Name,
    co.Destination, co.Status
FROM Customer c
JOIN Courier co
ON c.Customer_ID = co.Customer_ID;
```

6. Display courier and payment details

```
SELECT co.Tracking_ID, p.Amount,
    p.Payment_Mode, p.Payment_Status
FROM Courier co
JOIN Payment p
ON co.Courier_ID = p.Courier_ID;
```

7. Display courier tracking details

```
SELECT co.Tracking_ID, t.Location,
    t.Status, t.Updated_Date
FROM Courier co
JOIN Tracking t
ON co.Courier_ID = t.Courier_ID;
```

8. Display customer feedback

```
SELECT c.Name, co.Tracking_ID,
    f.Rating, f.Comments
FROM Feedback f
JOIN Customer c
ON f.Customer_ID = c.Customer_ID
JOIN Courier co
ON f.Courier_ID = co.Courier_ID;
```

9. Display delivered couriers

```
SELECT *FROM Courier

WHERE Status = 'Delivered';
```

10. Display couriers with weight greater than 5 kg

```
SELECT *

FROM Courier

WHERE Weight > 5;
```

9. SQL QUERIES & SCREENSHOTS

The following query set is prepared to satisfy the documentation requirement for a 30–40 query SQL section. The examples cover selection, filtering, joins, aggregation, subqueries, updates, deletion, and reporting. Exact outputs depend on the records inserted in the final project database.

Q1. Display all customers

```
SELECT * FROM CUSTOMER;
```

Output:

CUSTOMER_ID	NAME	PHONE	EMAIL	ADDRESS
1	Ravi Kumar	9876543210	ravi@gmail.com	Hyderabad
2	Priya Sharma	9876543211	priya@gmail.com	Vijayawada
3	Arjun Reddy	9876543212	arjun@gmail.com	Warangal
4	Sneha Rao	9876543213	sneha@gmail.com	Guntur
5	Kiran Kumar	9876543214	kiran@gmail.com	Nellore
6	Anjali Devi	9876543215	anjali@gmail.com	Tirupati
7	Rahul Varma	9876543216	rahul@gmail.com	Hyderabad
8	Divya Reddy	9876543217	divya@gmail.com	Karimnagar
9	Vikas Rao	9876543218	vikas@gmail.com	Nizamabad
10	Swathi	9876543219	swathi@gmail.com	Warangal
11	Manoj Kumar	9876543220	manoj@gmail.com	Khammam
12	Lakshmi	9876543221	lakshmi@gmail.com	Hyderabad
13	Suresh	9876543222	suresh@gmail.com	Secunderabad
14	Meena	9876543223	meena@gmail.com	Vijayawada
15	Tarun	9876543224	tarun@gmail.com	Guntur

Q2. Display customer names and their phone number

```
SELECT NAME, PHONE FROM CUSTOMER;
```

Output:

NAME	PHONE
Ravi Kumar	9876543210
Priya Sharma	9876543211
Arjun Reddy	9876543212
Sneha Rao	9876543213
Kiran Kumar	9876543214
Anjali Devi	9876543215
Rahul Varma	9876543216
Divya Reddy	9876543217
Vikas Rao	9876543218
Swathi	9876543219
Manoj Kumar	9876543220
Lakshmi	9876543221
Suresh	9876543222
Meena	9876543223
Tarun	9876543224

Q3. Find customers from Hyderabad

```
SELECT CUSTOMER_ID, NAME, PHONE, EMAIL
FROM CUSTOMER
WHERE ADDRESS = 'Hyderabad';
```

Output:

CUSTOMER_ID	NAME	PHONE	EMAIL
1	Ravi Kumar	9876543210	ravi@gmail.com
7	Rahul Varma	9876543216	rahul@gmail.com
12	Lakshmi	9876543221	lakshmi@gmail.com

Q4. Display all courier details

```
SELECT *FROM COURIER;
```

Output:

COURIER_ID	CUSTOMER_ID	RECEIVER_NAME	RECEIVER_PHONE	SOURCE	DESTINATION	WEIGHT	BOOKING_DATE	AGENT_ID
1001	1	Amit	8888800001	Hyderabad	Delhi	2.5	01-AUG-26	101
1002	2	Rahul	8888800002	Vijayawada	Hyderabad	1.2	02-AUG-26	102
1003	3	Neha	8888800003	Warangal	Chennai	3	03-AUG-26	103
1004	4	Sanjay	8888800004	Guntur	Bangalore	1.75	04-AUG-26	104
1005	5	Pooja	8888800005	Nellore	Hyderabad	4.5	05-AUG-26	105
1006	6	Ravi	8888800006	Tirupati	Mumbai	2	06-AUG-26	106
1007	7	Kavya	8888800007	Hyderabad	Pune	5.25	07-AUG-26	107
1008	8	Arun	8888800008	Karimnagar	Delhi	1.5	08-AUG-26	108
1009	9	Deepa	8888800009	Nizamabad	Chennai	3.75	09-AUG-26	109
1010	10	Vijay	8888800010	Warangal	Bangalore	2.25	10-AUG-26	110
1011	11	Nikhil	8888800011	Khammam	Hyderabad	6	11-AUG-26	111
1012	12	Harsha	8888800012	Hyderabad	Mumbai	2.8	12-AUG-26	112
1013	13	Anu	8888800013	Secunderabad	Pune	4	13-AUG-26	113
1014	14	Madhav	8888800014	Vijayawada	Delhi	3.2	14-AUG-26	114
1015	15	Keerthi	8888800015	Guntur	Chennai	1.8	15-AUG-26	115

Q5. Display couriers weighing more than 3 kg

```
SELECT COURIER_ID,SOURCE,DESTINATION,WEIGHT
FROM COURIER
WHERE WEIGHT > 3;
```

Output:

COURIER_ID	SOURCE	DESTINATION	WEIGHT
1005	Nellore	Hyderabad	4.5
1007	Hyderabad	Pune	5.25
1009	Nizamabad	Chennai	3.75
1011	Khammam	Hyderabad	6
1013	Secunderabad	Pune	4
1014	Vijayawada	Delhi	3.2

Q6. Display couriers from Hyderabad

```
SELECT COURIER_ID,
    CUSTOMER_ID,
    SOURCE,
    DESTINATION,
    WEIGHT
FROM COURIER
WHERE SOURCE = 'Hyderabad';
```

Output:

COURIER_ID	CUSTOMER_ID	SOURCE	DESTINATION	WEIGHT
1001	1	Hyderabad	Delhi	2.5
1007	7	Hyderabad	Pune	5.25
1012	12	Hyderabad	Mumbai	2.8

Q7. Display couriers going to Delhi

```
SELECT COURIER_ID,
    SOURCE,
    DESTINATION,
    WEIGHT
FROM COURIER
WHERE DESTINATION = 'Delhi';
```

Output:

COURIER_ID	SOURCE	DESTINATION	WEIGHT
1001	Hyderabad	Delhi	2.5
1008	Karimnagar	Delhi	1.5
1014	Vijayawada	Delhi	3.2

Q8. Display customer name and courier details

```
SELECT C.NAME AS CUSTOMER_NAME,
    CR.COURIER_ID,
    CR.SOURCE,
    CR.DESTINATION,
    CR.WEIGHT
FROM CUSTOMER C
JOIN COURIER CR
ON C.CUSTOMER_ID = CR.CUSTOMER_ID;
```

Output:

CUSTOMER_NAME	COURIER_ID	SOURCE	DESTINATION	WEIGHT
Ravi Kumar	1001	Hyderabad	Delhi	2.5
Priya Sharma	1002	Vijayawada	Hyderabad	1.2
Arjun Reddy	1003	Warangal	Chennai	3
Sneha Rao	1004	Guntur	Bangalore	1.75
Kiran Kumar	1005	Nellore	Hyderabad	4.5
Anjali Devi	1006	Tirupati	Mumbai	2
Rahul Varma	1007	Hyderabad	Pune	5.25
Divya Reddy	1008	Karimnagar	Delhi	1.5
Vikas Rao	1009	Nizamabad	Chennai	3.75
Swathi	1010	Warangal	Bangalore	2.25
Manoj Kumar	1011	Khammam	Hyderabad	6
Lakshmi	1012	Hyderabad	Mumbai	2.8
Suresh	1013	Secunderabad	Pune	4
Meena	1014	Vijayawada	Delhi	3.2
Tarun	1015	Guntur	Chennai	1.8

Q9. Display customer name, receiver name and destination

```
SELECT C.NAME AS CUSTOMER_NAME,
    CR.RECEIVER_NAME,
    CR.DESTINATION
FROM CUSTOMER C
JOIN COURIER CR
ON C.CUSTOMER_ID = CR.CUSTOMER_ID;
```

Output:

CUSTOMER_NAME	RECEIVER_NAME	DESTINATION
Ravi Kumar	Amit	Delhi
Priya Sharma	Rahul	Hyderabad
Arjun Reddy	Neha	Chennai
Sneha Rao	Sanjay	Bangalore
Kiran Kumar	Pooja	Hyderabad
Anjali Devi	Ravi	Mumbai
Rahul Varma	Kavya	Pune
Divya Reddy	Arun	Delhi
Vikas Rao	Deepa	Chennai
Swathi	Vijay	Bangalore
Manoj Kumar	Nikhil	Hyderabad
Lakshmi	Harsha	Mumbai
Suresh	Anu	Pune
Meena	Madhav	Delhi
Tarun	Keerthi	Chennai

Q10. Display courier and delivery-agent details

```
SELECT CR.COURIER_ID,
    CR.DESTINATION,
    A.NAME AS AGENT_NAME,
    A.PHONE AS AGENT_PHONE,
    A.AREA
FROM COURIER CR
JOIN DELIVERY_AGENT A
ON CR.AGENT_ID = A.AGENT_ID;
```

Output:

COURIER_ID	DESTINATION	AGENT_NAME	AGENT_PHONE	AREA
1001	Delhi	Ramesh	9000000001	Hyderabad
1002	Hyderabad	Srinivas	9000000002	Vijayawada
1003	Chennai	Mahesh	9000000003	Warangal
1004	Bangalore	Naresh	9000000004	Guntur
1005	Hyderabad	Rajesh	9000000005	Nellore
1006	Mumbai	Venu	9000000006	Tirupati
1007	Pune	Karthik	9000000007	Hyderabad
1008	Delhi	Prakash	9000000008	Karimnagar
1009	Chennai	Mohan	9000000009	Nizamabad
1010	Bangalore	Venkat	9000000010	Warangal
1011	Hyderabad	Sai	9000000011	Khammam
1012	Mumbai	Ajay	9000000012	Hyderabad
1013	Pune	Rohit	9000000013	Secunderabad
1014	Delhi	Vamsi	9000000014	Vijayawada
1015	Chennai	Teja	9000000015	Guntur

Q11.Display customer, courier and payment details

```
SELECT C.NAME AS CUSTOMER_NAME,
    CR.COURIER_ID,
    CR.DESTINATION,
    P.AMOUNT,
    P.PAYMENT_MODE,
    P.PAYMENT_STATUS
FROM CUSTOMER C
JOIN COURIER CR
ON C.CUSTOMER_ID = CR.CUSTOMER_ID
JOIN PAYMENT P
ON CR.COURIER_ID = P.COURIER_ID;
```

Output:

CUSTOMER_NAME	COURIER_ID	DESTINATION	AMOUNT	PAYMENT_MODE	PAYMENT_STATUS
Ravi Kumar	1001	Delhi	500	UPI	PAID
Priya Sharma	1002	Hyderabad	300	CARD	PAID
Arjun Reddy	1003	Chennai	650	CASH	PAID
Sneha Rao	1004	Bangalore	450	UPI	PAID
Kiran Kumar	1005	Hyderabad	800	CARD	PENDING
Anjali Devi	1006	Mumbai	550	CASH	PAID
Rahul Varma	1007	Pune	900	UPI	PAID
Divya Reddy	1008	Delhi	350	CARD	PAID
Vikas Rao	1009	Chennai	700	UPI	PENDING
Swathi	1010	Bangalore	475	CASH	PAID
Manoj Kumar	1011	Hyderabad	1000	CARD	PAID
Lakshmi	1012	Mumbai	600	UPI	PAID
Suresh	1013	Pune	750	CARD	PAID
Meena	1014	Delhi	680	CASH	PENDING
Tarun	1015	Chennai	420	UPI	PAID

Q12.Display courier tracking information

```
SELECT CR.COURIER_ID,
    CR.SOURCE,
    CR.DESTINATION,
    T.LOCATION,
    T.STATUS,
    T.UPDATED_DATE
FROM COURIER CR
JOIN TRACKING T
ON CR.COURIER_ID = T.COURIER_ID;
```

Output:

COURIER_ID	SOURCE	DESTINATION	LOCATION	STATUS	UPDATED_DATE
1001	Hyderabad	Delhi	Hyderabad	Booked	01-AUG-26
1002	Vijayawada	Hyderabad	Vijayawada	In Transit	02-AUG-26
1003	Warangal	Chennai	Warangal	Delivered	03-AUG-26
1004	Guntur	Bangalore	Guntur	In Transit	04-AUG-26
1005	Nellore	Hyderabad	Nellore	Booked	05-AUG-26
1006	Tirupati	Mumbai	Tirupati	Delivered	06-AUG-26
1007	Hyderabad	Pune	Hyderabad	In Transit	07-AUG-26
1008	Karimnagar	Delhi	Karimnagar	Booked	08-AUG-26
1009	Nizamabad	Chennai	Nizamabad	In Transit	09-AUG-26
1010	Warangal	Bangalore	Warangal	Delivered	10-AUG-26
1011	Khammam	Hyderabad	Khammam	In Transit	11-AUG-26
1012	Hyderabad	Mumbai	Hyderabad	Booked	12-AUG-26
1013	Secunderabad	Pune	Secunderabad	Delivered	13-AUG-26
1014	Vijayawada	Delhi	Vijayawada	In Transit	14-AUG-26
1015	Guntur	Chennai	Guntur	Booked	15-AUG-26

Q13. Display customers and their feedback

```
SELECT C.NAME AS CUSTOMER_NAME,
    F.RATING,
    F.COMMENTS
FROM CUSTOMER C
JOIN FEEDBACK F
ON C.CUSTOMER_ID = F.CUSTOMER_ID;
```

Output:

CUSTOMER_NAME	RATING	COMMENTS
Ravi Kumar	5	Excellent service
Priya Sharma	4	Good delivery
Arjun Reddy	5	Very fast
Sneha Rao	3	Average service
Kiran Kumar	4	Good packaging
Anjali Devi	5	Excellent
Rahul Varma	4	Good service
Divya Reddy	3	Delivery was late
Vikas Rao	5	Very good
Swathi	4	Satisfied
Manoj Kumar	5	Excellent delivery
Lakshmi	3	Average
Suresh	4	Good experience
Meena	5	Very good service
Tarun	4	Good delivery

Q14. Find the total number of customers

```
SELECT COUNT(*) AS TOTAL_CUSTOMERS
FROM CUSTOMER;
```

Output:

TOTAL_CUSTOMERS
15

Q15. Find the total number of couriers

```
SELECT COUNT(*) AS TOTAL_COURIERS
FROM COURIER;
```

Output:

TOTAL_COURIERS
15

Q16.Find the total payment amount

```
SELECT SUM(AMOUNT) AS TOTAL_PAYMENT
FROM PAYMENT;
```

Output:

TOTAL_PAYMENT
9125

Q17.Find the average courier weight

```
SELECT AVG(WEIGHT) AS AVERAGE_WEIGHT
FROM COURIER;
```

Output:

AVERAGE_WEIGHT
3.033333333333333333333333333333333333

Q18.Find the maximum courier weight

```
SELECT MAX(WEIGHT) AS MAX_WEIGHT
FROM COURIER;
```

Output:

MAX_WEIGHT
6

Q19.Find the minimum courier weight

```
SELECT MIN(WEIGHT) AS MIN_WEIGHT
FROM COURIER;
```

Output:

MIN_WEIGHT
1.2

Q20.Find the average feedback rating

```
SELECT AVG(RATING) AS AVERAGE_RATING
FROM FEEDBACK;
```

Output:

AVERAGE_RATING
4.2

Q21. Find the total payment amount.

```
SELECT SUM(AMOUNT) AS TOTAL_PAYMENT
FROM PAYMENT;
```

Output:

TOTAL_PAYMENT
9125

Q22. Find the number of customers in each city.

```
SELECT ADDRESS,
    COUNT(*) AS TOTAL_CUSTOMERS
FROM CUSTOMER
GROUP BY ADDRESS;
```

Output:

ADDRESS	TOTAL_CUSTOMERS
Tirupati	1
Nizamabad	1
Guntur	2
Nellore	1
Karimnagar	1
Vijayawada	2
Hyderabad	3
Warangal	2
Secunderabad	1
Khammam	1

Q23. Find the number of couriers handled by each delivery agent.

```
SELECT A.AGENT_ID,
    A.NAME,
    COUNT(CR.COURIER_ID) AS TOTAL_COURIERS
FROM DELIVERY_AGENT A
LEFT JOIN COURIER CR
ON A.AGENT_ID = CR.AGENT_ID
GROUP BY A.AGENT_ID, A.NAME;
```

Output:

AGENT_ID	NAME	TOTAL_COURIERS
101	Ramesh	1
104	Naresh	1
112	Ajay	1
114	Vamsi	1
105	Rajesh	1
102	Srinivas	1
103	Mahesh	1
107	Karthik	1
106	Venu	1
108	Prakash	1
110	Venkat	1
113	Rohit	1
115	Teja	1
111	Sai	1
109	Mohan	1

Q24. Find customers who have booked at least one courier.

```
SELECT DISTINCT C.CUSTOMER_ID,
    C.NAME
FROM CUSTOMER C
JOIN COURIER CR
ON C.CUSTOMER_ID = CR.CUSTOMER_ID;
```

Output:

CUSTOMER_ID	NAME
8	Divya Reddy
9	Vikas Rao
13	Suresh
2	Priya Sharma
11	Manoj Kumar
3	Arjun Reddy
4	Sneha Rao
5	Kiran Kumar
12	Lakshmi
14	Meena
1	Ravi Kumar
15	Tarun
6	Anjali Devi
7	Rahul Varma
10	Swathi

Q25. Find couriers whose payment amount is greater than 500.

```
SELECT CR.COURIER_ID,
    CR.SOURCE,
    CR.DESTINATION,
    P.AMOUNT
FROM COURIER CR
JOIN PAYMENT P
ON CR.COURIER_ID = P.COURIER_ID
WHERE P.AMOUNT > 500;
```

Output:

COURIER_ID	SOURCE	DESTINATION	AMOUNT
1003	Warangal	Chennai	650
1005	Nellore	Hyderabad	800
1006	Tirupati	Mumbai	550
1007	Hyderabad	Pune	900
1009	Nizamabad	Chennai	700
1011	Khammam	Hyderabad	1000
1012	Hyderabad	Mumbai	600
1013	Secunderabad	Pune	750
1014	Vijayawada	Delhi	680

Q26. Display courier tracking details with customer names.

```
SELECT C.NAME AS CUSTOMER_NAME,
    CR.COURIER_ID,
    T.LOCATION,
    T.STATUS,
    T.UPDATED_DATE
FROM CUSTOMER C
JOIN COURIER CR
ON C.CUSTOMER_ID = CR.CUSTOMER_ID
JOIN TRACKING T
ON CR.COURIER_ID = T.COURIER_ID;
```

Output:

CUSTOMER_NAME	COURIER_ID	LOCATION	STATUS	UPDATED_DATE
Ravi Kumar	1001	Hyderabad	Booked	01-AUG-26
Priya Sharma	1002	Vijayawada	In Transit	02-AUG-26
Arjun Reddy	1003	Warangal	Delivered	03-AUG-26
Sneha Rao	1004	Guntur	In Transit	04-AUG-26
Kiran Kumar	1005	Nellore	Booked	05-AUG-26
Anjali Devi	1006	Tirupati	Delivered	06-AUG-26
Rahul Varma	1007	Hyderabad	In Transit	07-AUG-26
Divya Reddy	1008	Karimnagar	Booked	08-AUG-26
Vikas Rao	1009	Nizamabad	In Transit	09-AUG-26
Swathi	1010	Warangal	Delivered	10-AUG-26
Manoj Kumar	1011	Khammam	In Transit	11-AUG-26
Lakshmi	1012	Hyderabad	Booked	12-AUG-26
Suresh	1013	Secunderabad	Delivered	13-AUG-26
Meena	1014	Vijayawada	In Transit	14-AUG-26
Tarun	1015	Guntur	Booked	15-AUG-26

Q27. Find customers who have given a rating of 5.

```
SELECT C.NAME AS CUSTOMER_NAME,
    F.RATING,
    F.COMMENTS
FROM CUSTOMER C
JOIN FEEDBACK F
ON C.CUSTOMER_ID = F.CUSTOMER_ID
WHERE F.RATING = 5;
```

Output:

CUSTOMER_NAME	RATING	COMMENTS
Ravi Kumar	5	Excellent service
Arjun Reddy	5	Very fast
Anjali Devi	5	Excellent
Vikas Rao	5	Very good
Manoj Kumar	5	Excellent delivery
Meena	5	Very good service

Q28. Find the average rating given by customers.

```
SELECT AVG(RATING) AS AVERAGE_RATING
FROM FEEDBACK;
```

Output:

AVERAGE_RATING
4.2

Q29. Find the highest-rated courier.

```
SELECT CR.COURIER_ID,
       F.RATING,
       F.COMMENTS
FROM COURIER CR
JOIN FEEDBACK F
ON CR.COURIER_ID = F.COURIER_ID
WHERE F.RATING = (
   SELECT MAX(RATING)
   FROM FEEDBACK
);
```

Output:

COURIER_ID	RATING	COMMENTS
1001	5	Excellent service
1003	5	Very fast
1006	5	Excellent
1009	5	Very good
1011	5	Excellent delivery
1014	5	Very good service

Q30. Find couriers whose weight is greater than the average weight.

```
SELECT COURIER_ID,
       SOURCE,
       DESTINATION,
       WEIGHT
FROM COURIER
WHERE WEIGHT > (
   SELECT AVG(WEIGHT)
   FROM COURIER
);
```

Output:

COURIER_ID	SOURCE	DESTINATION	WEIGHT
1005	Nellore	Hyderabad	4.5
1007	Hyderabad	Pune	5.25
1009	Nizamabad	Chennai	3.75
1011	Khammam	Hyderabad	6
1013	Secunderabad	Pune	4
1014	Vijayawada	Delhi	3.2

11. CONCLUSION

The Courier Management System provides a structured relational database solution for efficiently managing customer, courier, payment, tracking, and delivery-agent information. The database uses primary keys, foreign keys, uniqueness rules, NOT NULL conditions, and CHECK constraints to maintain data integrity. Normalization up to Third Normal Form (3NF) reduces data redundancy and prevents update anomalies. The SQL implementation demonstrates data insertion, modification, retrieval, joins, filtering, aggregation, and reporting. Overall, the project provides a suitable DBMS-based foundation for efficient courier booking, delivery, and shipment management.

12. FUTURE ENHANCEMENTS

- Role-based access for administrators, staff, faculty, and other authorized users.
- SMS/email/app notifications for shipment status.
- Online payment gateway integration.
- Route optimization and delivery scheduling.
- Advanced reports and dashboards.
- Backup, recovery, audit logging, and stronger database security.
- Web and mobile application interfaces.

REFERENCES

1. Korth, H. F., Silberschatz, A., and Sudarshan, S., Database System Concepts, McGraw Hill.

2. Elmasri, R. and Navathe, S. B., Fundamentals of Database Systems, Pearson.

3. Ramakrishnan, R. and Gehrke, J., Database Management Systems, McGraw Hill.

4. Date, C. J., An Introduction to Database Systems, Pearson.

5. Connolly, T. and Begg, C., Database Systems: A Practical Approach to Design, Implementation, and Management, Pearson.

WEB REFERENCES

1.Oracle Database Documentation –

https://docs.oracle.com/en/database/

2.Oracle SQL Language Reference –

https://docs.oracle.com/en/database/oracle/oracle-database/19/sqlrf/

3.GeeksforGeeks – DBMS Tutorial –

https://www.geeksforgeeks.org/dbms/

Education.

5. Ramez Elmasri, Shamkant B. Navathe – *Fundamentals of Database Systems*, Pearson.

6. Raghu Ramakrishnan, Johannes Gehrke – *Database Management Systems*, McGraw-Hill Education.

7. C. J. Date – *An Introduction to Database Systems*, Pearson.